CardioSense AI: Cardiovascular Risk Prediction Using Machine Learning

Ankita
Registration No.: 26MAG0015
M.Sc. Artificial Intelligence and Machine Learning
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Arya Yadav
Registration No.: 26MAG0026
M.Sc. Artificial Intelligence and Machine Learning
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Divyanshu Kandpal
Registration No.: 26MAG0028
M.Sc. Artificial Intelligence and Machine Learning
Vellore Institute of Technology
Vellore, Tamil Nadu, India

***Abstract*—CardioSense AI is an educational machine-learning prototype designed to estimate the likelihood of a cardiovascular-disease label from selected health and lifestyle attributes. The project uses a publicly available cardiovascular dataset containing approximately 70,000 records and derives Body Mass Index (BMI), Mean Arterial Pressure (MAP), Pulse Pressure (PP), and age in years from original attributes. A web frontend communicates with a Python-based backend that performs model inference. This report describes the motivation, related work, dataset, feature engineering, model workflow, system architecture, evaluation methodology, limitations, and future work. Results must be measured using the exact model and evaluation split used by the team; metrics reported by the reference paper are not treated as results of this project. CardioSense AI is an academic prototype and is not intended to diagnose cardiovascular disease or replace professional medical assessment.**

***Index Terms*—Cardiovascular disease, machine learning, classification, feature engineering, health data, web application.**

I. Introduction

Cardiovascular diseases are an important public-health concern, and machine-learning methods are frequently studied for identifying patterns in structured health data. Their practical value depends on data quality, evaluation methodology, generalization, interpretability, and responsible communication of uncertainty.

CardioSense AI connects a trained classification model to a user-facing web application. Users enter selected health and lifestyle attributes, and the application returns a model-generated classification. The project combines data preprocessing, feature engineering, model deployment, backend API development, and frontend integration.

The output must be interpreted only in relation to the dataset and model used during development. A classification is not a clinical diagnosis, and performance on a public dataset does not establish clinical validity for real patients.

A. Problem Statement

The project addresses the implementation problem of building an accessible web interface that accepts structured cardiovascular-related attributes and passes them through a consistent preprocessing and machine-learning pipeline. The application must preserve the expected feature order and distinguish a trained-model response from any demonstration or fallback output.

B. Objectives

The objectives are to prepare cardiovascular data for supervised classification; derive BMI, MAP, PP, and age in years; train and package a classifier; integrate the model with a web interface through an API; evaluate performance using suitable metrics; and document system limitations.

C. Contributions

The contribution is an end-to-end educational implementation combining feature preparation, machine-learning inference, an API backend, and a web interface. Claims of improved accuracy or superiority over other models should be made only when supported by reproducible experiments.

II. Related Work

Omkari and Shaik proposed an Integrated Two-Layered Voting (TLV) framework for coronary artery disease prediction [1]. Their approach introduces BMI, MAP, and PP as engineered features and investigates feature selection with statistical methods, followed by classification using multiple machine-learning algorithms. Their reported results belong to their experimental setup and must not be attributed to CardioSense AI.

Jawalkar *et al.* studied early heart-disease prediction using supervised learning and a decision-tree-based random-forest approach with stochastic-gradient-boosting loss optimization [2]. Their work describes preprocessing, model training, and evaluation on a separate test dataset.

These papers motivate feature engineering and systematic evaluation. CardioSense AI focuses on implementing a practical web-based prediction workflow. The report therefore distinguishes methods discussed in the literature from components actually implemented in this project.

III. Dataset Description

The project uses a cardiovascular-disease dataset commonly distributed through Kaggle and containing approximately 70,000 records. The commonly distributed dataset contains demographic, examination, and lifestyle attributes with a binary target label. Confirm the exact dataset URL, version, and licence against the file actually used by the team before submission.

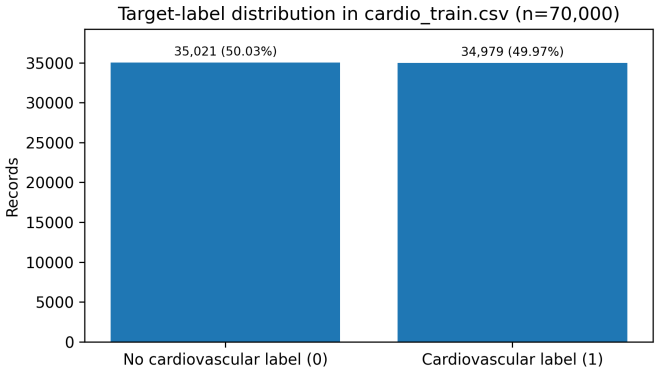


Fig. 1. Observed target-label counts in the supplied `cardio_train.csv` file. The file contains 70,000 records: 35,021 with `cardio`=0 and 34,979 with `cardio`=1.

TABLE I
ORIGINAL DATASET ATTRIBUTES

Attribute	Description
`id`	Record identifier; normally excluded from predictive inputs.
`age`	Age recorded in days in the commonly distributed dataset.
`gender`	Encoded gender attribute.
`height`	Height measurement.
`weight`	Weight measurement.
`ap_hi`	Systolic blood pressure.
`ap_lo`	Diastolic blood pressure.
`cholesterol`	Cholesterol category.
`gluc`	Glucose category.
`smoke`	Smoking indicator.
`alco`	Alcohol-consumption indicator.
`active`	Physical-activity indicator.
`cardio`	Binary target label.

A. Data Preparation

Describe the actual preparation script used in the project. Checks may include missing values, duplicates, implausible measurements, class balance, and inconsistent ranges. Document any rows removed, values corrected, or transformations applied. The identifier should not be used as a predictive feature. The age conversion policy must be consistent between training and inference.

IV. Feature Engineering

The project derives BMI, MAP, PP, and age in years. The derived features are motivated by the feature-engineering discussion in the reference literature [1]; their inclusion alone does not prove improved performance.

A. Body Mass Index

When height is measured in centimetres and weight in kilograms,

$$\mathrm{BMI} = \frac{\text{weight (kg)}}{(\text{height (m)})^2}. \tag{1}$$

Convert height from centimetres to metres before applying the equation.

B. Pulse Pressure and Mean Arterial Pressure

Pulse Pressure is calculated as

$$\mathrm{PP} = \mathrm{ap_hi} - \mathrm{ap_lo}. \tag{2}$$

An approximate Mean Arterial Pressure is calculated as

$$\mathrm{MAP} = \frac{\mathrm{ap_hi} + 2\,\mathrm{ap_lo}}{3}. \tag{3}$$

The documented model input order is shown in Table II. Ensure that it exactly matches the order expected by the final serialized model.

TABLE II
DOCUMENTED MODEL INPUT FEATURES

No.	Feature
1	BMI
2	MAP
3	PP
4	active
5	age_years
6	ap_hi
7	ap_lo
8	cholesterol
9	weight
10	gluc

V. Methodology

The workflow consists of data preparation, feature engineering, model training and selection, model serialization, backend inference, and frontend presentation. The exact algorithm and artifact used in deployment should be verified against the submitted source code.

A. Workflow

1) Load the dataset and inspect its columns and target distribution.
2) Apply the documented cleaning rules and exclude the identifier from predictive inputs.
3) Derive BMI, MAP, PP, and age in years.
4) Construct the feature matrix using the agreed feature order.
5) Split the data into training and evaluation partitions using a documented strategy.
6) Train candidate classifiers and select a model based on evaluation results and deployment constraints.
7) Save the selected model and any required preprocessing objects.
8) Load the artifact in the backend and validate incoming requests.
9) Return a prediction response to the frontend.

B. Classification Formulation

Represent each record by a feature vector

$$\mathbf{x} = [x_1, x_2, \ldots, x_{10}]. \tag{4}$$

A trained binary classifier maps the vector to

$$\hat{y} = f(\mathbf{x}), \qquad \hat{y} \in \{0, 1\}. \tag{5}$$

The output refers to the dataset target label and is not a definitive medical diagnosis.

C. Model Selection

Name the exact classifier used by the final deployment after checking the training script and backend. A compact model may be preferred for a hosting environment with limited memory, but deployment feasibility is not evidence that the compact model is more accurate.

VI. System Architecture

CardioSense AI uses a web frontend connected to a Python backend. The frontend submits user attributes to an API. The backend validates the request, arranges the features, invokes the saved model, and returns a response.

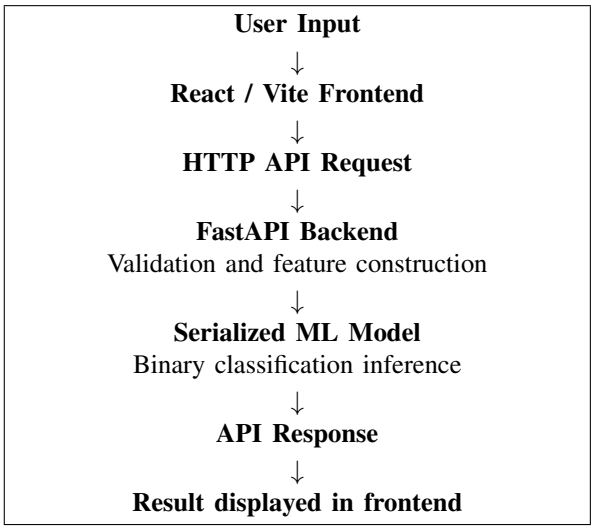


Fig. 2. Logical architecture of CardioSense AI. Update this figure if the final source code differs.

A. Frontend and Backend

The frontend is implemented using React and Vite. It collects input values and displays the API response. The backend is implemented using Python and FastAPI. It accepts input data, validates fields, prepares features in the required order, loads the deployed model, performs inference, and returns a structured response. Training and inference must use identical feature definitions and preprocessing.

B. Deployment

The frontend and backend are hosted separately and communicate through an API endpoint. Hosting availability does not validate predictive performance. Document relevant deployment constraints such as cold starts, service limits, dependency versions, and model memory use.

VII. Implementation

The model is stored as a serialized Python artifact and loaded by the backend. Insert the exact model filename, algorithm, and software versions used in the final deployment. Serialized models may depend on compatible Python and scikit-learn versions.

TABLE III
Historical Soft Voting Experiment Metrics (Not the Deployed Model)

Metric	Historical value (%)
Accuracy	73.24
Precision	75.01
Recall	68.82
F1-score	71.78
ROC-AUC	79.47

A. Inference Pseudocode

Listing 1. High-level inference workflow

```
INPUT: validated health attributes
OUTPUT: prediction and model status

1. Receive request from frontend
2. Validate required fields and ranges
3. Convert age to years if required
4. Compute BMI, MAP, and PP
5. Arrange features in training order
6. Invoke the deployed classifier
7. Return the prediction and model status
8. If inference fails, return an explicit error
9. Never label a fallback as a trained-model result
```

VIII. Evaluation Methodology

Evaluate the final model on a held-out test set or using a documented cross-validation strategy. Preprocessing and feature selection should be fitted only on training data where applicable. Reusing test data for model selection can produce an optimistic estimate.

A. Metrics

For binary classification,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{6}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \tag{7}$$

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{8}$$

Here, TP, TN, FP, and FN denote true positives, true negatives, false positives, and false negatives. ROC-AUC may be reported if appropriate prediction scores are available.

The metrics in Table III and Fig. 3 are historical experiment records and must be verified against the original training notebook before final submission. A tuned Random Forest experiment was separately recorded at 73.58% accuracy and 79.98% ROC-AUC; other metrics were not available in the records reviewed. No test-set size or 80:20 split is asserted here because the exact split configuration has not yet been verified.

The supplied dataset file was directly inspected for Fig. 1: it has 70,000 rows and 13 semicolon-separated columns. The target counts are 35,021 for class 0 (50.03%) and 34,979 for class 1 (49.97%). This descriptive count is computed from the supplied file, not inferred from a reference paper. It does

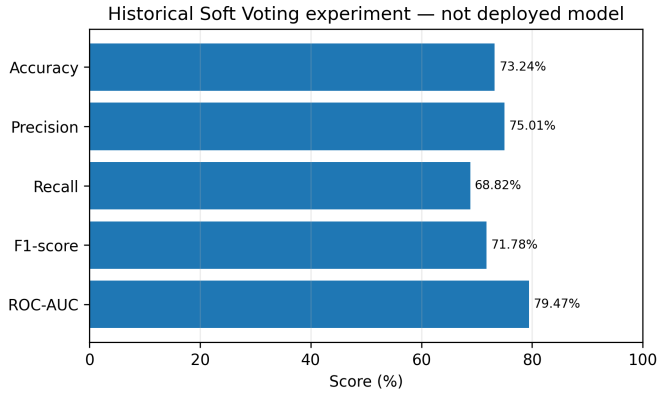


Fig. 3. Historical Soft Voting experiment scores recorded in project history. These values are not measurements of the currently deployed lightweight Decision Tree candidate.

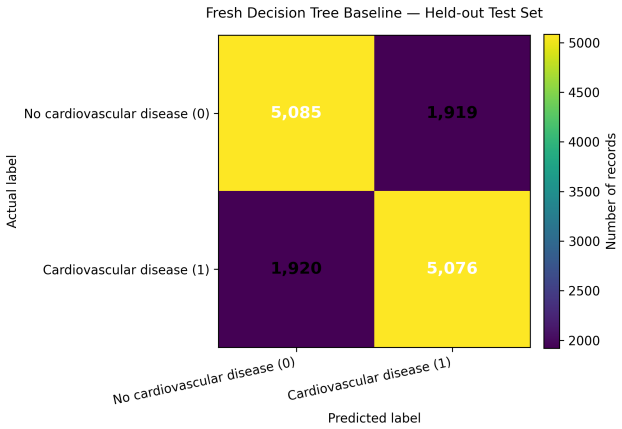


Fig. 4. Confusion matrix for a fresh Decision Tree baseline trained on the supplied dataset with a stratified 80:20 split (14,000 held-out records). This is a reproducibility baseline, not an evaluation of the deployed model artifact.

not establish the class distribution after any later cleaning or train/test splitting.

To provide a reproducible confusion-matrix example without misrepresenting the deployed artifact, a separate baseline was trained directly on the supplied CSV using a stratified 80:20 train/test split (random state 42). The baseline was a scikit-learn Decision Tree with maximum depth 8 and minimum leaf size 20, using the documented derived features. On the 14,000-row held-out partition, the confusion matrix was $[[5085, 1919], [1920, 5076]]$ with rows representing actual classes $[0, 1]$ and columns representing predicted classes $[0, 1]$. This gives 5,085 true negatives, 1,919 false positives, 1,920 false negatives, and 5,076 true positives. Its accuracy was 72.58%, precision 72.57%, recall 72.56%, and F1-score 72.56%. These are newly generated baseline results for this documented split only; they are not claimed to be the performance of the deployed model or the historical Soft Voting experiment.

IX. Website Pages and User Workflow

The deployed website is organized as a multi-page single-page application. Navigation is handled by React Router, while shared navigation and footer components provide consistent movement between pages. The production site is published as a GitHub Pages project site, so its base path is configured to match the repository path.

A. Landing Page

The landing page introduces CardioSense AI and provides an entry point to the assessment workflow. It is intended to explain the educational purpose of the application and guide users toward the available information pages. Text, illustrations, and navigation should describe the system as a prototype rather than a medical authority.

B. Risk Assessment Page

The assessment page collects supported health and lifestyle attributes through a form. The input set documented for the API includes age, height, weight, systolic and diastolic blood pressure, cholesterol, glucose, smoking, alcohol consumption, physical activity, and gender. Input validation should prevent missing or invalid values from being sent where validation rules are defined. The exact labels and permitted ranges should be verified against the final source code.

C. Results Page

The results page presents information returned by the prediction workflow. It should communicate the target classification in plain language and avoid implying that the result confirms or rules out disease. The result interface must also distinguish a response generated by the deployed model from a local demonstration fallback when the backend is unavailable.

D. Analytics Dashboard

The analytics dashboard provides descriptive visualizations related to the dataset and cardiovascular attributes. Potential views include target-label distribution, age, BMI, pulse pressure, mean arterial pressure, blood pressure, lifestyle indicators, and relationships between attributes. Only charts that are present in the final deployed site and supported by the actual dataset should be described as implemented. Any chart showing aggregate statistics should identify the dataset and should not be interpreted as a clinical conclusion about an individual.

E. Model Performance Page

The model-performance page is intended to present model comparison and evaluation information. Useful measures include accuracy, precision, recall, F1-score, and ROC-AUC where they have been measured correctly. Historical training experiments and the lightweight model currently used by the deployed service must be distinguished. If the current deployed candidate has not been independently evaluated on a leakage-safe holdout set, the page and report should state that limitation explicitly.

F. How It Works Page

The how-it-works page explains the sequence from user input through preprocessing and model inference to the displayed response. This explanation can describe the roles of feature engineering, the backend API, the saved estimator, and the frontend. It should not claim that every model experiment is part of the live inference path.

G. About Page

The about page describes the academic purpose, scope, and limitations of CardioSense AI. It can identify the project as an integration of data science, machine learning, and web development. It should also explain that model outputs are educational estimates based on a dataset and do not replace professional healthcare advice.

TABLE IV
WEBSITE ROUTE INVENTORY

Route	Purpose
`/`	Landing page and project introduction.
`/assess`	Health-attribute assessment form.
`/results`	Presentation of prediction information.
`/analytics`	Dataset and attribute visualizations.
`/model-performance`	Model comparison and evaluation information.
`/how-it-works`	Explanation of the processing workflow.
`/about`	Project background and educational scope.

X. API INTEGRATION AND DEPLOYMENT

The application separates the static frontend from the prediction service. This separation allows the user interface to be deployed independently of the Python environment used for model inference.

A. Prediction Request Lifecycle

The normal request lifecycle is:

1) The user opens the assessment page and enters supported attributes.
2) The frontend validates and organizes the input payload.
3) The API client sends a request to the backend prediction endpoint.
4) FastAPI validates and processes the request and invokes the loaded estimator when available.
5) The backend returns a structured JSON response.
6) The frontend renders the response and should identify the mode in which it was generated.

B. API Endpoints

The project documentation identifies the following endpoint roles. Verify exact response fields and HTTP status handling in the current backend source before final submission.

TABLE V
DOCUMENTED API ENDPOINT ROLES

Endpoint	Method	Purpose
`/model-status`	GET	Reports model/backend availability and status information.
`/predict`	POST	Receives assessment attributes and returns prediction information.

C. Hosting Configuration

The frontend is published using GitHub Pages and the backend is hosted separately on Render. The production frontend uses the project-site base path `/CardioSense-AI/` and connects to the Render API URL through a build-time environment setting. The GitHub Actions workflow builds the frontend and publishes the generated static files. During local development, the Vite development proxy can forward API calls to a local FastAPI server; that proxy is not a production hosting service.

D. Model Availability and Demonstration Fallback

The frontend API client has a fallback path for connectivity or backend errors. A fallback can keep the interface demonstrable, but it must never be presented as a successful inference from the trained model. A robust status message should distinguish at least a successfully loaded model, an unavailable backend, and demonstration mode. The fallback should be documented and tested separately from the trained classifier.

XI. MODEL EXPERIMENTS AND DEPLOYMENT CHOICE

During development, multiple classical classifiers were explored, including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, Multi-Layer Perceptron, a tuned Random Forest, and a Soft Voting ensemble. The final report must identify which of these experiments were actually completed and which artifact is loaded by the live service.

Historical experiment records list the Soft Voting model with accuracy 0.7324, precision 0.7501, recall 0.6882, F1-score 0.7178, and ROC-AUC 0.7947. A tuned Random Forest experiment was recorded with accuracy 0.7358 and ROC-AUC 0.7998. These values describe earlier experiments only; they must not be labelled as performance of the currently deployed lightweight candidate.

The deployed backend uses a compact Decision Tree candidate named `cardiosense_small_candidate.pkl`, according to the project documentation. This model was selected in response to the memory limits of the free hosting environment. A separate, leakage-safe evaluation of that exact artifact is still required before making performance claims about the deployed version.

TABLE VI
DISTINCTION BETWEEN HISTORICAL EXPERIMENTS AND DEPLOYMENT

Item	Status to Report
Soft Voting metrics	Historical experiment results; not the deployed candidate's metrics.
Tuned Random Forest	Historical experiment; report only the metrics actually recorded.
Compact Decision Tree	Documented live candidate used to meet deployment memory constraints.
Independent evaluation of live candidate	Required before asserting its generalization performance.
Clinical validation	Not established; outside the demonstrated project scope.

TABLE VII
SUGGESTED SYSTEM TEST CASES

Test	Expected Check
Page navigation	All listed routes open the intended page.
Form validation	Missing or invalid values are handled clearly.
API prediction	Valid request reaches the backend and returns structured output.
Model status	The UI reflects the current backend/model status.
Backend offline	Fallback is clearly labelled and not mistaken for model inference.
Feature consistency	Derived values and feature order match training.
Responsive layout	Forms, charts, and navigation remain usable on smaller screens.
Artifact loading	Backend reports a clear failure if the model cannot be loaded.

XII. TESTING AND QUALITY ASSURANCE

Testing should cover both the user interface and the backend inference path. A successful page load is not sufficient to demonstrate that the prediction service works correctly, and a successful API response is not by itself evidence of model quality.

A. Functional Testing

The team should verify that navigation reaches all documented pages, the assessment form rejects invalid input, the prediction request uses the correct fields, and result information is rendered without layout errors. The status endpoint should be checked when the backend is available and when it is unavailable. The fallback behavior should be tested to ensure that its output is clearly identified as a demonstration.

B. Model Pipeline Testing

The training and inference feature definitions must be compared directly. Tests should confirm the units used for height, the conversion of age to years, the formulae for BMI, MAP, and PP, and the exact order of model inputs. The model artifact should load under the pinned dependency versions. The backend should return a controlled error if the artifact cannot be loaded or inference fails.

C. Deployment Testing

The deployed frontend should be tested at its public GitHub Pages URL, and the backend status endpoint should be checked independently. Test cases should include valid inputs, invalid values, backend unavailability, delayed responses, and mobile-width layouts. Record the test date, environment, observed result, and any unresolved defects in the final submission.

XIII. SECURITY, PRIVACY, AND RESPONSIBLE USE

Health-related inputs should be treated as sensitive information even in an academic demonstration. The application should avoid unnecessary storage of entered data, should not expose credentials in frontend source code, and should use appropriate deployment configuration for API URLs and access controls. Logs should not retain identifiable health information unless there is a justified and documented requirement.

The application should explain the purpose and limitations of its output. It should not encourage users to make treatment decisions based on a model-generated label. Any future use with real patients would require expert oversight, privacy and security review, external validation, and relevant institutional and regulatory assessment.

XIV. DISCUSSION

CardioSense AI demonstrates the integration of structured-data preprocessing, feature engineering, machine-learning inference, API development, and frontend presentation. BMI, MAP, and PP are consistent with feature-engineering approaches discussed in the reference literature [1]. However, public datasets may contain measurement errors, population-specific patterns, and biases. A dataset label is not equivalent to a full clinical assessment, and reliable inference depends on consistent feature definitions, units, ordering, and software versions.

XV. LIMITATIONS AND ETHICAL CONSIDERATIONS

The system is an academic prototype and has not been established as a clinically validated diagnostic tool. Its output depends on the dataset, target label, model training procedure, and user-entered values. Incorrect or incomplete inputs can produce misleading results. Public datasets may not represent all populations or healthcare settings, and high performance on one dataset does not guarantee generalization.

The application should not be used to start, stop, or change treatment. Personal health information should be handled carefully, and any real-world use would require appropriate clinical evaluation, privacy review, security testing, and expert oversight.

XVI. Future Work

Potential extensions include reproducible comparison of classifiers, calibration analysis, threshold selection, explainability, subgroup performance evaluation, testing under dataset shift, stronger input validation, automated tests for training-inference consistency, and independent validation on an appropriate external dataset. These items should be described as future work unless implemented and tested.

XVII. Conclusion

This paper presented CardioSense AI, an educational prototype connecting a cardiovascular-disease classification model to a web-based interface. The project combines a structured data workflow, derived features, a Python API backend, and a frontend for entering attributes and displaying a model response. The final report should include measured results from the exact deployed artifact and a reproducible evaluation protocol. CardioSense AI is not a substitute for medical assessment, and further validation would be required before any clinical use.

Acknowledgment

The authors would like to express their sincere gratitude to their faculty members Dr. Padmakumari P. of Vellore Institute of Technology for their valuable guidance, encouragement, and support throughout the development of CardioSense AI.

References

[1] D. Y. Omkari and K. Shaik, "An Integrated Two-Layered Voting (TLV) Framework for Coronary Artery Disease Prediction Using Machine Learning Classifiers," *IEEE Access*, vol. 12, pp. 56275–56290, 2024, doi: 10.1109/ACCESS.2024.3389707.

[2] A. P. Jawalkar *et al.*, "Early prediction of heart disease with data analysis using supervised learning with stochastic gradient boosting," *Journal of Engineering and Applied Science*, vol. 70, art. no. 122, 2023, doi: 10.1186/s44147-023-00280-y.

[3] S. Ulianova, "Cardiovascular Disease Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset.